Javascript

What is JavaScript?

- ✓ JavaScript is one of the most used languages when it comes to building web applications as it allows developers to wrap HTML and CSS code in it to make web apps interactive.
- ✓ It enables the interaction of users with the web application. It is also used for making animations on websites
- ✓ *JavaScript* was initially created to "**make web pages alive**".
- ✓ The programs in this language are called *scripts*. They can be written right in a web page's HTML and run automatically as the page loads.
- ✓ Scripts are provided and executed as plain text. They don't need special preparation or compilation to run.
- ✓ It is lightweight and most commonly used as a part of web pages, whose implementations allow client-side script to interact with the user and make dynamic pages.
- ✓ It is an interpreted programming language with object-oriented capabilities.
- ✓ It is used both client-side and server-side to make web apps interactive
- ✓ Uses built-in browser DOM
- ✓ Extension of JavaScript file is .**js**

Why is it called JavaScript?
When JavaScript was created, it initially had another name: "**LiveScript**". But Java was very popular at that time, so it was decided that positioning a new language as a "younger brother" of Java would help.

But as it evolved, JavaScript became a fully independent language with its own specification called **ECMAScript**, and now it has no relation to Java at all.

ECMAScript, also known as JavaScript, is a programming language adopted by the European Computer Manufacturer's Association as a standard for performing computations in Web applications. We learn this ECMAScript in chapter 10.

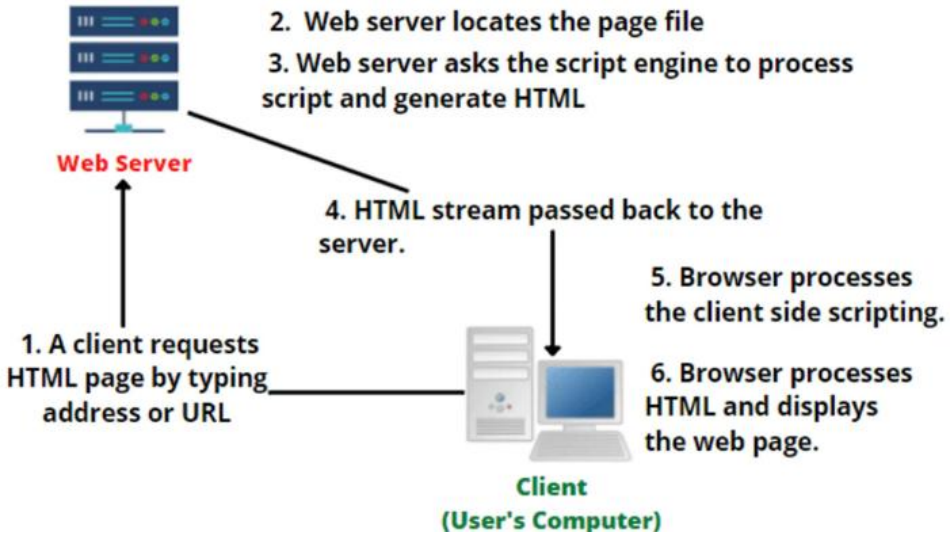
History of JavaScript

In 1993, **Mosaic**, the first popular web browser, came into existence. In the **year 1994**, **Netscape** was founded by **Marc Andreessen**. He realized that the web needed to become more dynamic. Thus, a 'glue language' was believed to be provided to HTML to make web designing easy for designers and part-time programmers. Consequently, in 1995, the company recruited **Brendan Eich** intending to implement and embed Scheme programming language to the browser. But, before Brendan could start, the company merged with **Sun Microsystems** for adding Java into its Navigator so that it could compete with Microsoft over the web technologies and platforms. Now, two languages were there: Java and the scripting language. Further, Netscape decided to give a similar name to the scripting language as Java's. It led to 'Javascript'. Finally, in May 1995, Marc Andreessen coined the first code of Javascript named '**Mocha**'. Later, the marketing team replaced the name with '**LiveScript**'. But, due to trademark reasons and certain

other reasons, in December 1995, the language was finally renamed to 'JavaScript'. From then, JavaScript came into existence.

Client-side scripting

- ✓ A client-side script is a tiny program (or collection of instructions) that is put into a web page. It is handled by the client browser rather than the web server.
- ✓ In other words, client-side scripting is a method for browsers to run scripts without having to connect to a server.
- ✓ The code runs on the client's computer's browser either while the web page is loading or after it has finished loading.
- ✓ Client-side scripting is mostly used for dynamic user interface components including pull-down menus, navigation tools, animation buttons, and data validation.
- ✓ The client refers to the script that runs on the user's computer system. It can either be integrated (or injected) into the HTML content or stored in a separate file (known as an external script).
- ✓ When the script files are requested, they are transmitted from the web server (or servers) to the client system. The script is run by the client's web browser, which subsequently displays the web page, including any visible script output.
- ✓ look at the diagram below.



Applications of Javascript Programming

- ✓ **Javascript** is one of the most widely used **programming languages** (Front-end as well as Back-end).
- ✓ It has it's presence in almost every area of software development
  - **Client side validation** - This is really important to verify any user input before submitting it to the server and Javascript plays an important role in validting those inputs at front-end itself.
  - **Manipulating HTML Pages** - Javascript helps in manipulating HTML page on the fly. This helps in adding and deleting any HTML tag very easily using javascript and

modify your HTML to change its look and feel based on different devices and requirements.

- **User Notifications** - You can use Javascript to raise dynamic pop-ups on the webpages to give different types of notifications to your website visitors.
- **Back-end Data Loading** - Javascript provides Ajax library which helps in loading back-end data while you are doing some other processing. This really gives an amazing experience to your website visitors.
- **Presentations** - JavaScript also provides the facility of creating presentations which gives website look and feel. JavaScript provides RevealJS and BespokeJS libraries to build a web-based slide presentations.
- **Server Applications** - Node JS is built on Chrome's Javascript runtime for building fast and scalable network applications. This is an event based library which helps in developing very sophisticated server applications including Web Servers.

Advantages of JavaScript:

The advantages of using JavaScript are the following:

- ✓ **Less server interaction** − You can validate user input before sending the page off to the server. This saves server traffic, which means less load on your server.
- ✓ **Immediate feedback to the visitors** − They don't have to wait for a page reload to see if they have forgotten to enter something.
- ✓ **Increased interactivity** − You can create interfaces that react when the user hovers over them with a mouse or activates them via the keyboard.
- ✓ **Richer interfaces** − You can use JavaScript to include such items as drag-and-drop components and sliders to give a Rich Interface to your site visitors.

Limitations of JavaScript:

We cannot treat JavaScript as a full-fledged programming language. It lacks the following important features:

- ✓ Client-side JavaScript does not allow the reading or writing of files. It has been kept for the security reason.
- ✓ JavaScript could not used for networking applications because there is no such support available.
- ✓ JavaScript doesn't have any multithreading or multiprocessor capabilities

Syntax of JavaScript:

- ✓ JavaScript can be implemented using JavaScript statements that are placed within the **<script>... </script>** HTML tags in a web page.
- ✓ You can place the **<script>** tags, containing your JavaScript, anywhere within your web page, but it is normally recommended that you should keep it within the **<head>** tags.
- ✓ The script tag takes two important attributes –

- **Language** − This attribute specifies what scripting language you are using. Typically, its value will be javascript. Although recent versions of HTML (and XHTML, its successor) have phased out the use of this attribute.
- **Type** − This attribute is what is now recommended to indicate the scripting language in use and its value should be set to "text/javascript".

✓ The <script> tag alerts the browser program to start interpreting all the text between these tags as a script.

```
<script language = "javascript" type = "text/javascript">
JavaScript code
</script>
```

There are 2 ways to use the javascript in the HTML file:

Internal JavaScript: JavaScript can be added directly to the HTML file by writing the code inside the <script> tag . We can place the <script> tag either inside <head> or the <body> tag according to the need.

Example:

```
<html>
  <head>
    <script language = "javascript" type = "text/javascript">
        document.write("Hello World!")
        console.log(“Hello!”)
    </script>
  </head>
</html>
```

External JavaScript: The other way is to write JavaScript code in another file having a .js extension and then link the file inside the <head> or <body> tag of the HTML file in which we want to add this code.

Example:

j1.html

```
<html>
  <head>
    <script src="j1.js"></script>
  </head>
</html>
```

j1.js

```
document.write("This is written in external file")
```

JavaScript in <body> and <head> sections together:

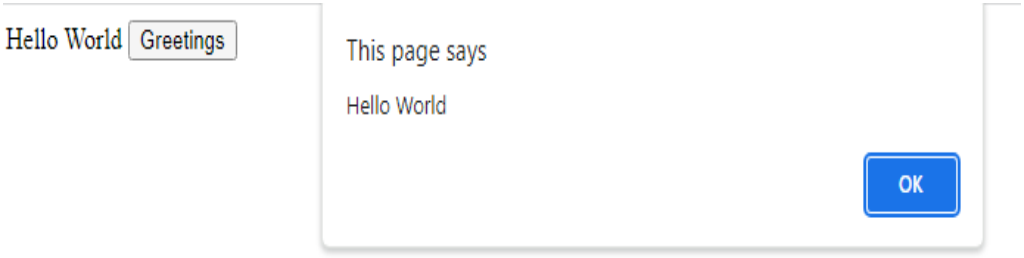
```
<!DOCTYPE html>

<head>

   <script type="text/javascript" language="javascript">
```

```
        function sayHello(){
                    alert("Hello World")
            }
    </script>
</head>
<body>
    <script type="text/javascript" language="javascript">
            document.write("Hello World")
    </script>
    <input type="button" onClick="sayHello()" value="Greetings"/>
</body>
</html>
```

Output:



Variables

- ✓ **Variables are Containers for Storing Data**
- ✓ **Variables** are the building blocks of any programming language.
- ✓ In JavaScript, variables can be used to store reusable values.
- ✓ The values of the variables are allocated using the assignment operator("=").

JavaScript Identifiers
JavaScript variables must have unique names. These names are called **Identifiers**.
There are some basic rules to declare a variable in JavaScript:

- ✓ Name must start with a letter (a to z or A to Z), underscore(_), or dollar($) sign.
- ✓ After first letter we can use digits (0 to 9), for example value1.
- ✓ JavaScript variables are case sensitive, for example x and X are different variables.
- ✓ A variable name cannot be a reserved keyword.

JavaScript is a dynamically typed language so the type of variables is decided at runtime. Therefore, there is no need to explicitly define the type of a variable. We can declare variables in JavaScript in four ways:

- Automatically
- var
- let
- const

All three keywords do the basic task of declaring a variable but with some differences Initially, all the variables in JavaScript were written using the var keyword but in ES6 the keywords let and const were introduced.

JavaScript local variable

- ✓ A JavaScript local variable is declared inside block or function.
- ✓ It is accessible within the function or block only.

For example:

```
<script>
function abc(){
var x=10;//local variable
}
abc();//calling JavaScript function
</script>
```

Output:

10

JavaScript global variable

- ✓ A **JavaScript global variable** is accessible from any function.
- ✓ A variable i.e. declared outside the function or declared with window object is known as global variable. **For example:**

```
<script>
var d=200;//global variable
function a(){
document.writeln(d);
}
document.writeln(d);
a();//calling JavaScript function
</script>
```

Output:

200 200

Automatically

In this first example, x, y, and z are undeclared variables. They are automatically declared when first used:

```
x = 5;
y = 6;
z = x + y;
```

var

- ✓ The **var** is the oldest keyword to declare a variable in JavaScript.
- ✓ It has the Global scoped or function scoped which means variables defined outside the function can be accessed globally, and variables defined inside a particular function can be accessed within the function.

1. The below code example explains the use of the var keyword to declare the variables in JavaScript.

```
var a = 10
function f1() {
```

```
    var b = 20
    console.log(a, b)
}
f1();
console.log(a);
```

Output:
10 20
10

2. The below example explains the behavior of var variables when declared inside a function and accessed outside of it.

```
function f1() {
    // It can be accessible anywhere within this function
    var a = 10;
    console.log(a)
}
f1();

// A cannot be accessible outside of function
console.log(a);
```

Output:
10
ReferenceError: a is not defined

3. The below code re-declare a variable with same name in the same scope using the var keyword, which gives no error in the case of var keyword.

```
var a = 10
// User can re-declare variable using var
var a = 8
// User can update var variable
a = 7
console.log(a);
```

Output:

7

4. The below code explains the initialized the variables after accessing.

```
console.log(a);
var a = 10; //initialized the variable after accessing
```

Output:

Undefined

let

- ✓ The let keyword is an improved version of the var keyword.
- ✓ It is introduced in the ES6 or EcmaScript 2015.
- ✓ These variables has the block scope.
- ✓ It can't be accessible outside the particular code block ({block}).

1. The below code declares the variable using the let keyword.

```
let a = 10;
function f() {
   let b = 9
   console.log(b);
   console.log(a);
}
f();
console.log(b);

Output:
9
10
Uncaught ReferenceError: b is not defined
```

2. The below code explains the behaviour of let variables when they are re-declared in the same scope.

```
let a = 10
// It is not allowed
let a = 10
// It is allowed
a = 10

Output:
Uncaught SyntaxError:Identifier 'a' has already been declared
```

3.The below code explains the behavior of let variable when it is re-declared in the different scopes.

```
let a = 10
function f1() {
   let a = 9
   console.log(a) // It prints 9
}
f1()
console.log(a) // It prints 10
```

Output:

9

10

4. The below code explains the initialized the variables after accessing.

```
console.log(a);
let a = 10; //initialized the variable after accessing
```

Output:

Uncaught **ReferenceError**: Cannot access 'a' before initialization

const

- ✓ The const keyword has all the properties that are the same as the let keyword, except the user cannot update it and have to assign it with a value at the time of declaration.
- ✓ These variables also have the block scope.
- ✓ It is mainly used to create constant variables whose values cannot be changed once they are initialized with a value.

1.This code tries to change the value of the const variable.

```
const a = 10;
function f() {
   a = 9
   console.log(a)
}
f();
```

Output:

TypeError:**Assignment to constant variable.**

Note:

The variables declared with var and let are mutable that is their value can be changed but variables declared using const are immutable.

var	let	const
The scope of a *var* variable is functional or global scope.	The scope of a *let* variable is block scope.	The scope of a *const* variable is block scope.
It can be updated and re-declared in the same scope.	It can be updated but cannot be re-declared in the same scope.	It can neither be updated or re-declared in any scope.
It can be declared without initialization.	It can be declared without initialization.	It cannot be declared without initialization.
It can be accessed without initialization as its default value is “undefined”.	It cannot be accessed without initialization otherwise it will give ‘referenceError’.	It cannot be accessed without initialization, as it cannot be declared without initialization.

Data Types

- ✓ JavaScript provides different data types to hold different types of values.
- ✓ **There are two types of data types in JavaScript.**
  1) **Primitive data type**
  2) **Non-primitive (reference) data type**
- ✓ JavaScript is a dynamic type language, means you don't need to specify type of the variable because it is dynamically used by JavaScript engine.
- ✓ You need to use var, let or const here to specify the data type.
- ✓ It can hold any type of values such as numbers, strings etc.

For example:

```
var a=20;//holding number

var b="PPSU";//holding string
```

Primitive data types

- ✓ The predefined data types provided by JavaScript language are known as primitive data types. Primitive data types are also known as in-built data types.
- ✓ primitive data types are a set of basic data types from which all other data types are constructed

Below are the primitive data types.

1) **String** represents sequence of characters e.g. "hello"
   - x = "PPSU"; //x is a String
2) **Number** represents numeric values e.g. 100
   - // With decimals:
     let x1 = 24.00;
     // Without decimals:
     let x2 = 24;
3) **Boolean** represents Boolean value either **false** or **true**
   - let x = 5;
     let y = 5;
     let z = 6;
     (x == y) // Returns true
     (x == z) // Returns false

   - ✓ == is the comparison operator. It will only return true if both values are equivalent after coercing their types to the same type.
   - ✓ === is a more strict comparison operator often called the identity operator. It will only return true if both the type and value of the operands are the same.
4) **Undefined** represents undefined value
   - let name; // Value is undefined, type is undefined
5) **Null** represents null i.e. no value at all
   - let name = null; // value is null

All primitive types, except null, can be tested by the typeof operator. typeof null returns "object", so one has to use === null to test for null.

Non-primitive data types

- ✓ Non-primitive data types are called reference types because they refer to objects.
- ✓ The main difference between primitive and non-primitive data types are: Primitive types are predefined (already defined) in Java.
- ✓ Non-primitive types are created by the programmer and is not defined by Java (except for String).

The non-primitive data types are as follows:

1) **Object** represents instance through which we can access members
    - JavaScript objects are written with curly braces {}.
    - Object properties are written as name:value pairs, separated by commas.
        - const person = {firstName:"ABC", lastName:"DEF", age:30};

```
<script>
  const person = {firstName:"Khushbu", lastName:"Patel"};
  document.write(person.firstName);
</script>
```

Output:
Khushbu

2) **Array** represents group of similar values
    - Arrays are a special kind of objects, with numbered indexes.
    - An array can hold many values under a single name, and you can access the values by referring to an index number.
    - It is a common practice to declare arrays with the const keyword.
    - JavaScript arrays are written with square brackets.
    - Array items are separated by commas.
    - The following code declares (creates) an array called fruits, containing three items (car names):
        - const fruits = ["grapes", "mango", "watermelon"];
    - ✓ There are two main ways to create arrays in JavaScript:
        - you can use a literal, or
        - you can use a constructor.
    - ✓ The **literal** looks like this:
        - **const myArray = [];**
    - ✓ and can be defined with or without any array elements. The one above doesn't have any elements yet.
    - ✓ The **constructor** looks like this and uses the new keyword on the Array object.
        - **const myArray = new Array();**

The length Property

The length property of an array returns the length of an array (the number of array elements).

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
let length = fruits.length;
```

The length property is always one more than the highest array index.

Accessing the First Array Element

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
let fruit = fruits[0]
```

Accessing the Last Array Element

```
const fruits = ["Banana", "Orange", "Apple", "Mango"];
let fruit = fruits[fruits.length - 1];
```

Example: Display all elements of an array in new line.

```
<script type="text/javascript" language="javascript">
    var emp =["abc", "pqr", "xyz"];
    for(i=0; i<emp.length; i++) {
        document.write(emp[i]+"<br>");
    }
</script>
```

Output:

abc
pqr
xyz

Above example using Array "new" keyword

Syntax

```
var arrayname = new Array();
```

```
<script>
var emp = new Array();
emp[0] = "abc";
emp[1] = "xyz";
emp[2] = "pqr";
for(i=0; i<emp.length; i++)
{  document.write(emp[i]+"<br>"); }
</script>
```

Output:
abc
pqr
xyz

Javascript Array constructor
Example

```
<html>
<body>
<script type="text/javascript" language="javascript">
var i;
var emp = new Array("abc", "xyz", "pqr");
for(i=0; i<emp.length; i++)
{
document.write(emp[i]+"<br>");
}
</script>
</body>
</html>
```

Output:
abc
xyz
pqr

Example: Multiply each elements of an array by 2 and display in new line.

```
<script>
  var num = [2,4,3,1,5];
  for(i=0; i<num.length; i++)
  {
    let mul = num[i]*2;
  document.write("multiplication is: "+ mul+"<br>");
  }
  </script>
```

Output:
multiplication is: 4
multiplication is: 8
multiplication is: 6
multiplication is: 2
multiplication is: 10

When to Use Arrays. When to use Objects.

- JavaScript does not support associative arrays.
- You should use **objects** when you want the element names to be **strings (text)**.
- You should use **arrays** when you want the element names to be **numbers**.

Example

```
var a = undefined;
var b = "test";
var c=true;
var d=5;
let e=null;
document.write(typeof(a)+"<br>")
document.write(typeof(b)+"<br>")
document.write(typeof(c)+"<br>")
document.write(typeof(d)+"<br>")
document.write(typeof(e)+"<br>")
```

Output:

```
undefined
string
boolean
number
object
```

Example:

```
<html>
<body>
<script type="text/javascript" language="javascript">
var x = 16 + 4 +"xyz";  //Treats 16 and 4 as numbers, until it reaches "xyz".
var y = "xyz" + 16 + 4;  // the first operand is a string, all operands are treated as strings.
document.write(x+"<br>");
document.write(y);
</script>
</body>
</html>
```

Output:

```
20xyz
xyz164
```

Inbuilt Objects

JavaScript provides various built-in objects. Each object contains properties and methods. Some of the built-in objects in Javascript are:

- Array
- Date
- Math
- String
- Number

Here, we will learn about Math, Date, String objects

Math object

The Math object provides a number of methods to work with Number values.

Methods available on *Math* object are:

Method	Description
max(a,b)	Returns largest of a and b
min(a,b)	Returns least of a and b
round(a)	Returns nearest integer
ceil(a)	Rounds up. Returns the smallest integer greater than or equal to a
floor(a)	Rounds down. Returns the largest integer smaller than or equal to a
pow(a,b)	Returns a^b
abs(a)	Returns absolute value of a
random()	Returns a pseudo random number between 0 and 1
sqrt(a)	Returns square root of a

```
<script type="text/javascript" language="javascript">
document.write("MATH FUNCTIONS");
document.write("<br/>Square root : "+Math.sqrt(9));
document.write("<br/>Absolute : "+Math.abs(-1));
document.write("<br/>Ceil : "+Math.ceil(1.4));
document.write("<br/>Negative Ceil : "+Math.ceil(-1.4));
document.write("<br/>Floor : "+Math.floor(1.7));
document.write("<br/>Negative Floor : "+Math.floor(-1.7));
document.write("<br/>Power: "+Math.pow(3,2));
```

```
document.write("<br/>Maximum: "+Math.max(3,2,2.4,3.5));
document.write("<br/>Minimum: "+Math.min(5,5.5));
</script>
```

MATH FUNCTIONS
Square root : 3
Absolute : 1
Ceil : 2
Negative Ceil : -1
Floor : 1
Negative Floor : -2
Power: 9
Maximum: 3.5
Minimum: 5

Date object

At times you there will be a need to access the current date and time and also past and future date and times. JavaScript provides support for working with dates and time through the ***Date*** object.

Date objected is created using the ***new*** **operator** and one of the *Date's* constructors. Current date and time can be retrieved as shown below:

```
var today = new Date( );
```

Methods available on *Date* object are:

Method	**Description**
Date()	Creates a *Date* object with the current date and time of the browser's PC
Date("Month,dd, yyyy hh:mm:ss")	This creates a *Date* object with the specified string
Date("Month dd, yyyy")	This creates a *Date* object with the specified string
Date(yy,mm,dd,hh,mm,ss)	This creates a *Date* object with the specified string
Date(yy,mm,dd)	This creates a *Date* object with the specified date

Date(milliseconds)	Creates a *Date* object with the date value represented by the number of milliseconds since midnight on Jan 1, 1970
getFullYear()	Returns the 4-digit year component of the date.
getMonth()	Returns the month component of the date. Returns the month (0 to 11) of a date. January =0, February = 1, ... upto December=11
getDate()	Returns the date component of the date.
getDay()	Returns the day component of the date. ["Sunday","Monday","Tuesday","Wednesday","Thursday","Friday","Saturday"] **(0-6)** Sunday-0 to Saturday- 6
getTime()	returns the number of milliseconds since January 1, 1970 00:00:00.
getHours()	Returns the hours component of the date
getMinutes()	Retrieves the minutes component of the date
getSeconds()	Retrieves the seconds component of the date
getMilliseconds()	Returns the milliseconds component of the date
setFullYear()	Sets the year component of the date using a 4-digit number
setMonth()	Sets the month component of the date. January =0, February = 1, ... upto December=11
setTime()	The setTime() method sets a date and time by adding or subtracting a specified number of milliseconds to/from midnight January 1, 1970.
setDate()	Sets the day-of-month component of the date
setHours()	Set the hours component of the date
setMinutes	Set the minutes component of the date
setSeconds()	Sets the seconds component of the date
setMilliseconds()	Sets the milliseconds component of the date

Example:

```
<html>

<body>

<script type="text/javascript" language="javascript">

var d1 = new Date();

document.write(d1+"<br>");

document.write(d1.getDay()+"<br>");

document.write(d1.getMonth()+"<br>");

document.write(d1.getDate()+"<br>");

document.write(d1.getFullYear()+"<br>");

document.write(d1.getHours()+"<br>");

document.write(d1.getMinutes()+"<br>");

document.write(d1.getSeconds()+"<br>");

</script>

</body>

</html>
```

Wed Jan 31 2024 11:40:16 GMT-0800 (Pacific Standard Time)
3
0
31
2024
11
40
16

Write a JavaScript that uses function to calculate how many days are left in your birthday.

```
<html>
<head>
```

```
<script>
function daysdifference(date1, date2)
{
   one_day =24*60*60*1000; // one day time in milliseconds (86400000)
   date1_ms= date1.getTime(); //get time in milliseconds ( 0 hours, 0 minutes, 0 seconds starting from January 1, 1970 to Sat Feb 10 2024)
   date2_ms= date2.getTime(); //get time in milliseconds ( 0 hours, 0 minutes, 0 seconds starting from January 1, 1970 to current date)
   difference_ms = Math.abs(date1_ms-date2_ms);
   return(Math.round(difference_ms/one_day));
}
var d1 = new Date();
d1.setDate(10); // set date
d1.setMonth(1); //set month
// now d1 = Sat Feb 10 2024
var d2 = new Date(); //current date
document.write("No of days left : "+ daysdifference(d1,d2));
</script>
</head>
</html>
```

String object

- ✓ A string is a collection of characters.
- ✓ JavaScript provides various properties and methods to work with String objects.
- ✓ One most frequently used property on String objects is *length*. The *length* property gives the number of characters in the given string. Consider the following example:

```
var str = “Hello World”;
var len = str.length;
```

The value stored in the *len* variable will be 11 which the number of characters in the string.

Methods available on *String* objects are:

Method	Description
charAt(index)	Returns the character at specified index
charCodeAt(index)	Returns the unicode value of the character at specified index
concat()	Combines two or more strings

indexOf(searchtext, index)	Searches for the specified string from the beginning of the string.
lastIndexof(searchtext, index)	Searches for the specified string from the end of the string
replace(expr, new-string)	Replaces the old string with the specified new string
substr(start, length)	Returns a substring having length characters from a specified index **start** Required.The start position.First character is at index 0. If start is greater than the length, substr() returns "". If start is negative, substr() counts from the end of the string. **Length** Optional.The number of characters to extract. If omitted, it extracts the rest of the string
substring(start, end)	The substring() method extracts characters, between two indices (positions), from a string, and returns the substring. **start** Required.Start position.First character is at index 0. **end** Optional.End position (up to, but not including). If omitted: the rest of the string.
toUpperCase()	Returns the string after converting all characters to upper case
toLowerCase()	Returns the string after converting all characters to lower case
trim()	The trim() method removes whitespace from both sides of a string.
split()	splits a string into an array of substrings.

Example:

```
<html>
<body>
<script type="text/javascript" language="javascript">
document.write("<br/>STRING FUNCTIONS");
x="hello";
y=" World";
```

```
x1="this
is
demo";
y1="is";
document.write("<br/>char at 0th position : "+x.charAt(0));
document.write("<br/>Unicode value at 0th position: "+x.charCodeAt(0));
document.write("<br/>concated string : "+x.concat(y));
document.write("<br/>first index occ: "+x1.indexOf(y1)); document.write("<br/>last index occ:
"+x1.lastIndexOf(y1));
document.write("<br/>replace: "+x.replace("hell",y));
document.write("<br/>split: "+x.split("l"));
document.write("<br/>split: "+x1.split(" "));
document.write("<br/>sub str: "+x1.substr(2,5));
document.write("<br/>sub string: "+x1.substring(2,6));
document.write("<br/>sub string: "+x.toUpperCase());
document.write("<br/>sub string: "+y.toLowerCase());
</script> </body> </html>
```

STRING FUNCTIONS
char at 0th position : h
Unicode value at 0th position: 104
concated string : hello World
first index occ: 2
last index occ: 5
replace: Worldo
split: he,,o
split: this,is,demo
sub str: is is
sub string: is i
sub string: HELLO
sub string: world

Dec	Hex	Char
48	30	0
49	31	1
50	32	2
51	33	3
52	34	4
53	35	5
54	36	6
55	37	7
56	38	8
57	39	9

Dec	Hex	Char
65	41	A
66	42	B
67	43	C
68	44	D
69	45	E
70	46	F
71	47	G
72	48	H
73	49	I
74	4A	J
75	4B	K
76	4C	L
77	4D	M
78	4E	N
79	4F	O
80	50	P
81	51	Q
82	52	R
83	53	S
84	54	T
85	55	U
86	56	V
87	57	W
88	58	X
89	59	Y
90	5A	Z

Dec	Hex	Char
97	61	a
98	62	b
99	63	c
100	64	d
101	65	e
102	66	f
103	67	g
104	68	h
105	69	i
106	6A	j
107	6B	k
108	6C	l
109	6D	m
110	6E	n
111	6F	o
112	70	p
113	71	q
114	72	r
115	73	s
116	74	t
117	75	u
118	76	v
119	77	w
120	78	x
121	79	y
122	7A	z

Example:
Write a JS that find position of first occurrence of vowel ‘a’ and last occurrence of vowel ‘a’ in the given word “ajanta” also return the string between them.

```
<html>
<head>
<script type="text/javascript">
z="ajanta";
x=z.indexOf("a");
y=z.lastIndexOf("a");
document.write("<br/>first index occ: "+x);
document.write("<br/>last index occ: "+y);
document.write("<br/>sub string: "+z.substring(x+1,y));
</script>
</head>
<body>
</body>
</html>
```

Functions: Syntax, Calling function on some event

- ✓ A function is a group of reusable code which can be called anywhere in your program.
- ✓ This eliminates the need of writing the same code again and again.
- ✓ Functions allow a programmer to divide a big program into a number of small and manageable functions.

- ✓ Before we use a function, we need to define it.
- ✓ A JavaScript function is defined with the function keyword, followed by a **name**, followed by parentheses **()**.
- ✓ Function names can contain letters, digits, underscores, and dollar signs (same rules as variables).
- ✓ The parentheses may include parameter names separated by commas: ***(parameter1, parameter2, ...)***
- ✓ The code to be executed, by the function, is placed inside curly brackets: **{}**

```
function functionName(Parameter1, Parameter2, ...)
{
   // Function body
}
```

- ✓ A function in JavaScript is similar to a procedure—a set of statements that performs a task or calculates a value, but for a procedure to qualify as a function, it should take some input and return an output where there is some obvious relationship between the input and the output.
- ✓ To use a function, you must define it somewhere in the scope from which you wish to call it.

Function Invocation:

- Triggered by an event (e.g., a button click by a user).
- When explicitly called from JavaScript code.
- Automatically executed, such as in self-invoking functions.

Example of function without passing parameter

```
<head>
    <script>
        function fun() {
            document.write ("Hello! How are you?");
        }
    </script>
</head>
<body> <input type = "button" onclick = "fun()" value = "Click"></body>
```

Example of function with passing parameter

```
<head>
    <script>
        function fun(f_name,l_name) {
            document.write ("Hello " + f_name  + " " + l_name + " !");
        }
    </script>
</head>
<body> <input type = "button" onclick = "fun('ABC','XYZ')" value = "Click"></body>
```

JavaScript return statement

- ✓ It is used to return a particular value from the function. The function will stop the execution when the return statement is called and return a specific value.
- ✓ The return statement should be the last statement of the function because the code after the return statement won't be accessible.

- ✓ We can return any value i.e. Primitive value (Boolean, number and string, etc) or object type value (function, object, array, etc) by using the return statement.

```
return value;
```

Example:

```
function Product(a, b) {
// Return the product of a and b
return a * b;
};
console.log(Product(6, 10));
```

Assigned function to the variable

```
    var a = function ( x, y ) {
      return x + y;
    }
    let result = a(3, 5);
document.write(result);
```

or

```
var res = fun(12, 30);
function fun(x,y)
{
return x * y;
}
document.write(res);
```

JavaScript if…else statement:

Very often when you write code, you want to perform different actions for different decisions. You can use conditional statements in your code to do this.

In JavaScript we have the following conditional statements:

- Use if to specify a block of code to be executed, if a specified condition is true
- Use else to specify a block of code to be executed, if the same condition is false
- Use else if to specify a new condition to test, if the first condition is false

Syntax

```
if (condition) {
        // block of code to be executed if the condition is true
    }
```

Example:

```
<html>
<body>
        <script type="text/javascript" language="javascript">
        var age= 20;
        if(age>=18)
        {
            document.write("You are eligible for voting");
        }
        else
        {
            document.write("You are not eligible for voting");
        }
</script>
</body>
</html>
```

Output:

You are eligible for voting

The else if Statement

Use the else if statement to specify a new condition if the first condition is false.

Syntax

```
if (condition1) {
  //  block of code to be executed if condition1 is true
} else if (condition2) {
  //  block of code to be executed if the condition1 is false and condition2 is true
} else {
  //  block of code to be executed if the condition1 is false and condition2 is false
}
```

Example:

```
<html>
<body>
<script type="text/javascript" language="javascript">
var sub="FSD-1";
if(sub=="FSD-1")
{
 document.write("PKP");
}
else if(sub=="FCSP-1")
{
 document.write("MVP");
}
else if(sub=="DE")
{
 document.write("UMM");
}
else if(sub=="PS")
{
 document.write("PCS");
```

```
}
else
{
 document.write("Wrong Choice!!");
}
</script></body></html>
```

Output:

PKP

Loops: for, while, do…while

Types of Loops

1. Entry Controlled (In this type of loop, the test condition is tested before entering the loop body) – while, for

2. Exit Controlled (In this type of loop the test condition is tested or evaluated at the end of the loop body. Therefore, the loop body will execute at **least once**, irrespective of whether the test condition is true or false) **–** do… while

- **While Loop:** is a control flow statement that allows the code to be executed repeatedly based on the given Boolean condition.
  Syntax:

  while(expression)

  {

  Statement(s) to be executed **if expression is true**

  }

Example

```
<html><body>
<script type="text/javascript" language="javascript">
 var count = 0;
```

```
while (count < 10)
{                document.write("Current Count : " + count + "<br>");
                 count++;
}
</script></body></html>
```

Output:

```
Current Count : 0
Current Count : 1
Current Count : 2
Current Count : 3
Current Count : 4
Current Count : 5
Current Count : 6
Current Count : 7
Current Count : 8
Current Count : 9
```

- **For Loop:** is used to iterate the elements for a fixed number of times. JavaScript for loop is used if the number of the iteration is known.

  Syntax:

  ```
  for (initialization; test condition; iteration statement)
  {
      Statement(s) to be executed if test condition is true
  }
  ```

Example

```
<html>
<body>
<script type="text/javascript" language="javascript">
 for(count = 0; count < 10; count++)
 {
                 document.write("Current Count : " + count );
                 document.write("<br >");
 }
</script>
</body>
</html>
```

Output:

```
Current Count : 0
Current Count : 1
Current Count : 2
```

Current Count : 3
Current Count : 4
Current Count : 5
Current Count : 6
Current Count : 7
Current Count : 8
Current Count : 9

- **Do..while Loop:** It is a control flow statement that executes a block of code at least once, and then repeatedly executes the block or not depending on a given Boolean condition at the end of the block.
  Synatax:

  do
  {
   Statement(s) to be executed if test condition is true
  }
  while(expression);

Example

```
<html>
<body>
<script type="text/javascript" language="javascript">
 var count = 0;
 do
 {
   document.write("Current Count : " + count );
   document.write("<br>");
   count++;
 }
 while(count<10);
</script></body></html>
```

Output:

Current Count : 0
Current Count : 1
Current Count : 2
Current Count : 3
Current Count : 4
Current Count : 5
Current Count : 6
Current Count : 7
Current Count : 8
Current Count : 9

Break statement: The break statement is used to jump out of a loop. It can be used to "jump out" of a switch() statement. It breaks the loop and continues executing the code after the loop.

Example

```
<html>
<body>
<script type="text/javascript" language="javascript">
 var x = 0;
 while (x < 10)
     {
                if (x == 5)
                {
                   break;
                 }
                x = x + 1;
                document.write( x + "<br>");
      }
</script></body></html>
```

Output:

1
2
3
4
5

Continue statement: The continue statement "jumps over" one iteration in the loop. It breaks iteration in the loop and continues executing the next iteration in the loop**.**

Example

```
<html>
<head>
<script type="text/javascript" language="javascript">
var x = 0;
 while (x < 10)
 {
                x = x + 1;
                if (x == 5)
 {
                continue;
 }
 document.write( x + "<br />");
 }
```

```
</script>
</head>
</html>
```

Output:

1
2
3
4
6
7
8
9
10

- **Switch Case:**

JavaScript switch statement is used with a break or default keyword (optional and both can be used together also).

break: This keyword is used to break out of the switch block. This stops the execution inside the code block.

default: This keyword is used to specify a piece of code if no case matches the given condition. There can be only one default keyword in a switch statement.

Syntax

```
switch (expression)
{
 case condition 1: statement(s)
 break;

 case condition 2: statement(s)
 break;
 ...

 case condition n: statement(s)
 break;

 default: statement(s)
}
```

Example

```
<html>
<head>
<script type="text/javascript" language="javascript">
var grade = 'A';
```

```
 switch (grade)
 {
 case 'A': document.write("Good job<br />");
 break;

 case 'B': document.write("Pretty good<br />");
 break;

 case 'C': document.write("Passed<br />");
 break;

 case 'D': document.write("Not so good<br />");
 break;

 case 'F': document.write("Failed<br />");
 break;

 default: document.write("Unknown grade<br />");
 }
</script></head></html>
```

Output:
Good job

Pop up Boxes: Alert, Confirm, Prompt

Alert Box
An alert dialog box is mostly used to give a warning message to the users. For example, if one input field requires to enter some text but the user does not provide any input, then as a part of validation, you can use an alert box to give a warning message.

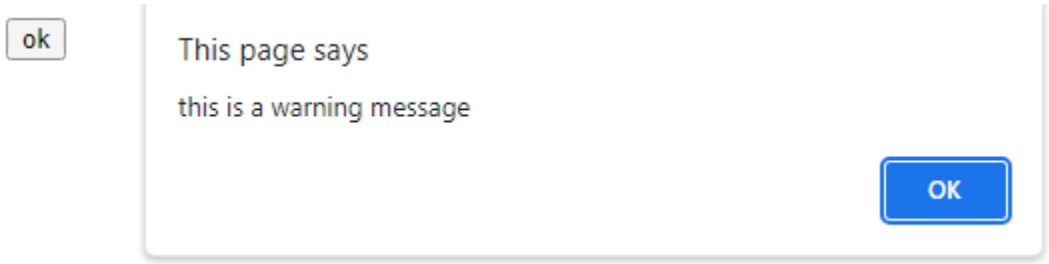
Nonetheless, an alert box can still be used for friendlier messages. Alert box gives only one button "OK" to select and proceed.

Example

```
<html>
<body>
<script>
function warn()
{
   alert("this is a warning message");
   document.write("Warning!!!!");    //It will be printed after closing alert box.//
```

```
}
</script>
</head>
<body>
   <input type="button" value="ok" onclick="warn()"/>
</body>
</html>
```

Output:



Confirmation Box

A confirmation dialog box is mostly used to take user's consent on any option. It displays a dialog box with two buttons: OK and Cancel.

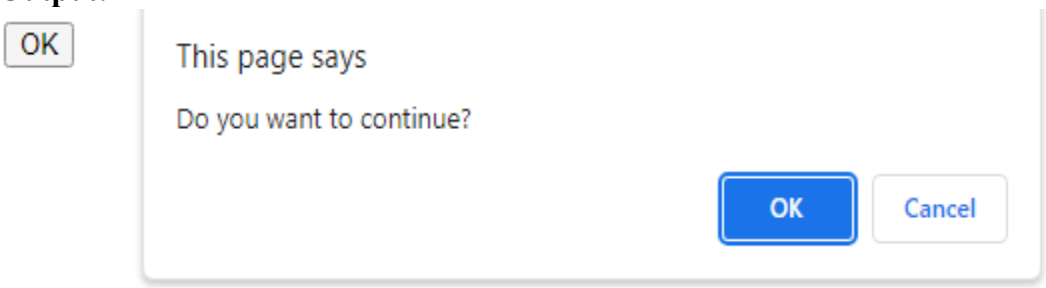
If the user clicks on the OK button, the window method confirm() will return true. If the user clicks on the Cancel button, then confirm() returns false. You can use a confirmation dialog box as follows.

Example

```
<html>
<head>
<script type="text/javascript">
function conf()
{
ret = confirm("Do you want to continue?");          //It returns value as true or false//
if(ret==true)
{
   document.write("user wants");
   return true;
}
else
{
   document.write("user does not want");
   return false;
}
}
```

```
</script>
</head>
<body>
   <input type="button" onclick="conf()" value="OK"/>
</body>
</html>
```

Output:



Prompt Dialog Box

The prompt dialog box is very useful when you want to pop-up a text box to get user input. Thus, it enables you to interact with the user. The user needs to fill in the field and then click OK.

This dialog box is displayed using a method called **prompt()** which takes two parameters:

(i) a label which you want to display in the text box and

(ii) a default string to display in the text box.

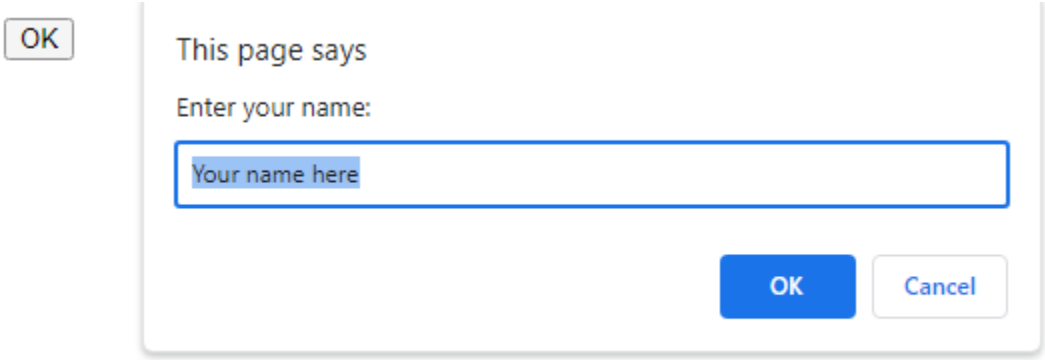
This dialog box has two buttons: OK and Cancel. If the user clicks the OK button, the window method prompt() will return the entered value from the text box. If the user clicks the Cancel button, the window method prompt() returns null.

Example:

```
<html>
<head>
<script type="text/javascript">
function getval()
{
   var ret = prompt("Enter your name: ", "Your name here");
   document.write("you entered "+ret);
}
</script>
</head>
<body>
<input type="button" onclick="getval()" value="OK"/>
```

```
</body>
</html>
```

Output:



Tasks:

PROGRAM:1 <!--1+ x/1! + x2/2! + x3/3!.... -->

Solution:

```
<html><head>
<script type="text/javascript">
function fact(n)
{  if(n==1)
    return(1);
        else
        {
                factorial=n*fact(n-1);
                return factorial;
        }
}
</script></head>
<body>
<script type="text/javascript">
x=3;sum=0;
```

```
for(n=1;n<=4;n++)
{
        sum=sum+Math.pow(x,n)/fact(n);
}
total=1+sum;
document.write(total);
</script></body></html>
```

Output:

16.375

PROGRAM-2 TWO DIGIT SEPARATE AND MULTIPLY

Solution:

```
<html>
<head>
  <script type="text/javascript">
     n=23;mul=0;
     m=n%10;
     n=parseInt(n/10);
      mul=Math.pow(n,m);
      document.write("answer is :" + mul);
  </script>
</head>
</html>
```

Output:

answer is :8

PROGRAM-3 LUCKY NUMBER

Solution:

```
<html>
<head>
```

```
</head>
<body>
<script type="text/javascript">
  n=101;m=n;
  while(n>9)
  {  sum=0;
     while(n>0)
     {
       i=n%10;
       sum=sum+i;
             n=parseInt(n/10);
           }
           n=sum;
         }
         if(n==1)
           document.write( m + " is lucky no" );
         else
           document.write( m+ "  is not lucky number");
   </script>
</body>
</html>
```

Output:

101 is not lucky no

PROGRAM-6 BUBBLE SORT

Solution:

```
<html>
<head>
<script type="text/javascript">
arr=new Array();
```

```
for(i=0;i<5;i++)
{
    arr[i]=parseInt(prompt("enter value"));
 }
// arr=[30,67,12,4,29];
document.write("elements before swap: "+ "</br>");
for(i=0;i<arr.length;i++)
 document.write(arr[i]+ "</br>");
  for(i=0;i<arr.length;i++)
 { for(j=0;j<arr.length-1;j++)
   { if(arr[i]<arr[j])
      { temp=arr[i];
                arr[i]=arr[j];
                      arr[j]=temp;
                     }
}
}
document.write("elements after swap: "+ "</br>");
for(i=0;i<arr.length;i++){
     document.write(arr[i] + "</br>");}
 </script> </head> </html>
```

Output:

elements before swap:
10
5
8
16
58
elements after swap:
5
8
10
16
58

Document Object Model (DOM): document object

With the HTML DOM, JavaScript can access and change all the elements of an HTML document.

With the object model, JavaScript gets all the power it needs to create dynamic HTML:

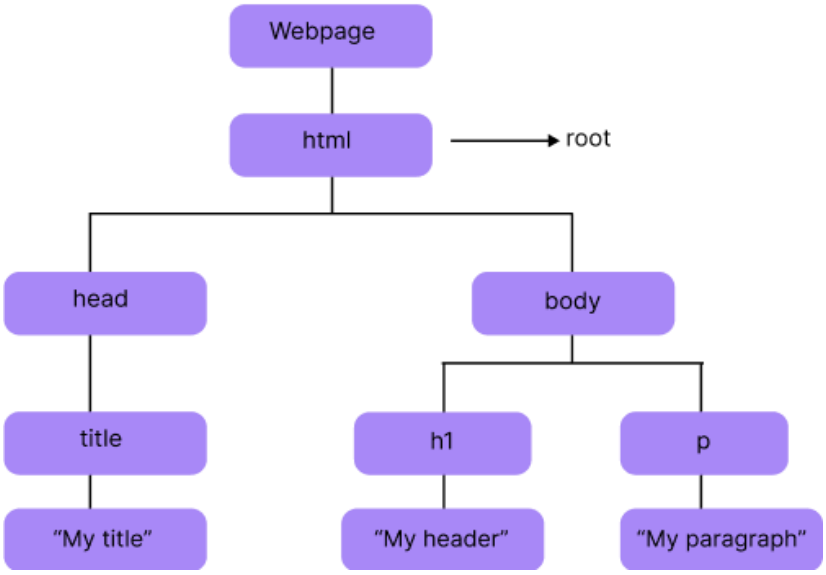
- JavaScript can change all the HTML elements in the page
- JavaScript can change all the HTML attributes in the page
- JavaScript can change all the CSS styles in the page
- JavaScript can remove existing HTML elements and attributes
- JavaScript can add new HTML elements and attributes
- JavaScript can react to all existing HTML events in the page
- JavaScript can create new HTML events in the page

What is the HTML DOM?

The HTML DOM is a standard object model and programming interface for HTML. It defines:

- The HTML elements as objects
- The properties of all HTML elements
- The methods to access all HTML elements
- The events for all HTML elements

In other words: The HTML DOM is a standard for how to get, change, add, or delete HTML elements.



DOM methods: write(), getElementById(), getElementsByClassName(), getElementsByTagName()

- HTML DOM methods are actions you can perform on HTML elements.
- HTML DOM properties are values that you can set or change of HTML elements.

- In DOM, all HTML elements are defined as objects.

The innerHTML Property

The easiest way to get the content of an element is by using the inner HTML property. The inner HTML property is useful for getting or replacing the content of HTML elements. The inner HTML property can be used to get or change any HTML element, including <html> and <body>.

Traversing and Modifying DOM Tree: innerHTML, attribute, setting style Changing HTML Elements

Property	Description
element.innerHTML = new html content	Change the inner HTML of an element
element.attribute = new value	Change the attribute value of an HTML element
element.style.property = new style	Change the style of an HTML element
Method	**Description**
element.setAttribute(attribute, value)	Change the attribute value of an HTML element

Example:

```
<html>
<body>
<p id="demo"></p>
      //<p id="demo">hi</p>//  //hi will be replaced with "Hello World!"
<script type="text/javascript" language="javascript">
      document.getElementById("demo").innerHTML = "Hello World!";
</script>
</body>
</html>
```

Output:

Hello World!

Note: <script> should be after <p> tag, since id is defined in <p> tag. If <script> tag is before <p> tag then <p> tag will give output as it is.

The getElementById Method

The most common way to access an HTML element is to use the id of the element. In the example above the getElementById method used id="demo" to find the element.

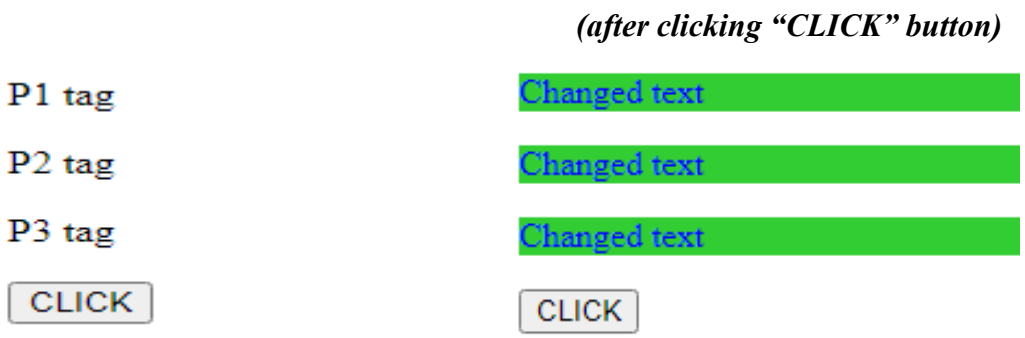
Method	Description
write()	writes the given string on the document.
getElementById(id)	returns the element having the given id value.
getElementsByTagName(name)	returns all the elements having the given tag name.
getElementsByClassName(name)	returns all the elements having the given class name.

Example: (using TagName)

```
<html>
<head>
<script type="text/javascript">
    function cls()
   {
         cl = document.getElementsByTagName("p");
         for(i=0; i<cl.length;i++)
       {
         cl[i].innerHTML = "Changed text";
         cl[i].style.color = "blue";
         cl[i].style.backgroundColor = "limegreen";
         }
```

```
    }
</script></head>
<body>
        <p>P1 tag</p>
        <p>P2 tag</p>
        <p>P3 tag</p>
        <input type="button" onclick="cls();" value="CLICK"/>
        </body>
</html>
```

Output:



Example: (using ClassName)

```
<html>
<head>
<script type="text/javascript">
  function cls()
  {
        cl = document.getElementsByClassName("same");
        for(i=0; i<cl.length;i++)
        {
          cl[i].innerHTML = "Changed text";
          cl[i].style.color = "blue";
```

```
            cl[i].style.backgroundColor = "limegreen";
        }
    }
</script></head>
<body>
        <h1 class="same">H1 tag</h1>
        <p class="same">P tag</p>
        <pre class="same">Pre tag</pre>
        <input type="button" onclick="cls();" value="CLICK"/>
</body>
</html>
```

Output:

(after clicking "CLICK" button)

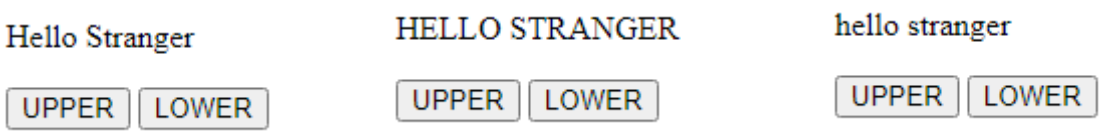


Example:

```
<html>
<head>
<script>
function upr()
{
cl = document.getElementsByTagName("p");
    for(i=0; i<cl.length;i++)
```

```
    {
        cl[i].style.textTransform = "upperCase";
    }
}
function lwr()
{
cl = document.getElementsByTagName("p")
    for(i=0; i<cl.length;i++)
    {
        cl[i].style.textTransform = "lowerCase";
    }
}
</script></head>
<body>
    <p>Hello Stranger</p>
    <input type="button" onclick="upr()" value="UPPER"/>
    <input type="button" onclick="lwr()" value="LOWER"/>
</body>
</html>
```

Output:



Note:

- In ClassName and TagName, it returns value in the form of array. So, for update in it you have to use for loop.
- while assigning css to a tag, we must use camel case for css property. For eg, fontsize should not be written like font-size like normal css. It should be written like fontSize in camelCase.

Example: (setAttribute)

```
<html>
<body>
    <p id="i1">para1</p>
    <p id="i2">para2</p>
    <script>
        var i1=document.getElementById("i1")
        var i2=document.getElementById("i2")
        i1.align="center"
        i2.setAttribute("align","right")
    </script>
</body>
</html>
```

Output:

para1

para2

Event Handling with JavaScript

The change in the state of an object is known as an Event. In html, there are various events which represents that some activity is performed by the user or by the browser. When JavaScript code is included in HTML, js react over these events and allow the execution. This process of reacting over the events is called Event Handling. Thus, js handles the HTML events via Event Handlers.

For example, when a user clicks over the browser, add js code, which will execute the task to be performed on the event.

- When a user clicks the mouse
- When a web page has loaded
- When an image has been loaded

- When the mouse moves over an element
- When an input field is changed
- When an HTML form is submitted
- When a user strokes a key

Mouse events:

Event Handler	Description
onclick	When mouse click on an element
onmouseover	When the cursor of the mouse comes over the element
onmouseout	When the cursor of the mouse leaves an element
onmousedown	When the mouse button is pressed over the element
onmouseup	When the mouse button is released over the element
onmousemove	When the mouse movement takes place.

Keyboard events:

Event Handler	Description
onkeyup	when the user releases a key on the keyboard
onkeydown	when the user presses a key on the keyboard. (for ctrl, shift, function etc. keys)
onkeypress	when the user presses a key on the keyboard (for letters, numbers, symbols, when some output is there on screen)

Form events:

Event Handler	Description
onblur	When the focus is away from a form element (clicks out of input field after clicking in input field)
onchange	When the user modifies or changes the value of a form element
onfocus	When the user focuses on an element (clicks in input field)
onsubmit	When the user submits the form

Example: (OnClick)

```
<html>
<head>
<script type="text/javascript">
   function change(id)
```

```
    {
        id.innerHTML = "Done";
    }
</script>
</head>
<body>
        <h1 onclick="change(this)">CLICK ON ME</h1>
</body>
</html>
```

Output:



(After clicking)

Example: (onchange, onfocus)

```
<html>
<body>
<script type="text/javascript">
    function upper(id)
    {
        id.value=id.value.toUpperCase();
    }
</script>
<!-- press enter after changing text to see uppercase  -->
    <input type="text" value="hello" onchange="upper(this)"/>
    <input type="text" value="hello" onfocus="this.style.backgroundColor='yellow'"/>
```

```
</body>
</html>
```

Output:



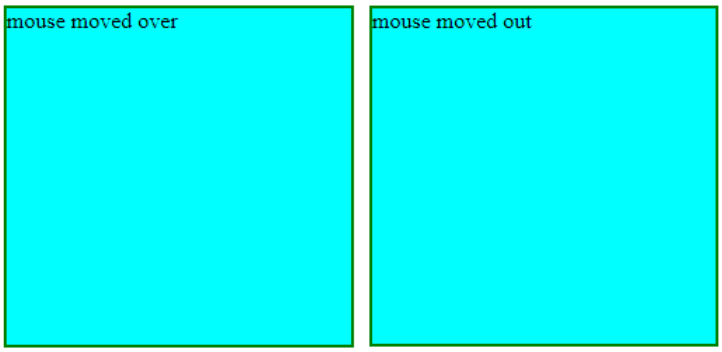
(after change and focus)



Example: (onmouseout, onmouseover)

```
<html>
<body>
<script type="text/javascript">
function fun(id)
{
   id.innerHTML = "mouse moved over";
}
function fun2(id)
{
   id.innerHTML = "mouse moved out";
}
</script>
<div onmouseover="fun(this)" onmouseout="fun2(this)" style="width:50%; height:40%; background-color:cyan; border:2px solid green;">
</body>
</html>
```

Output:

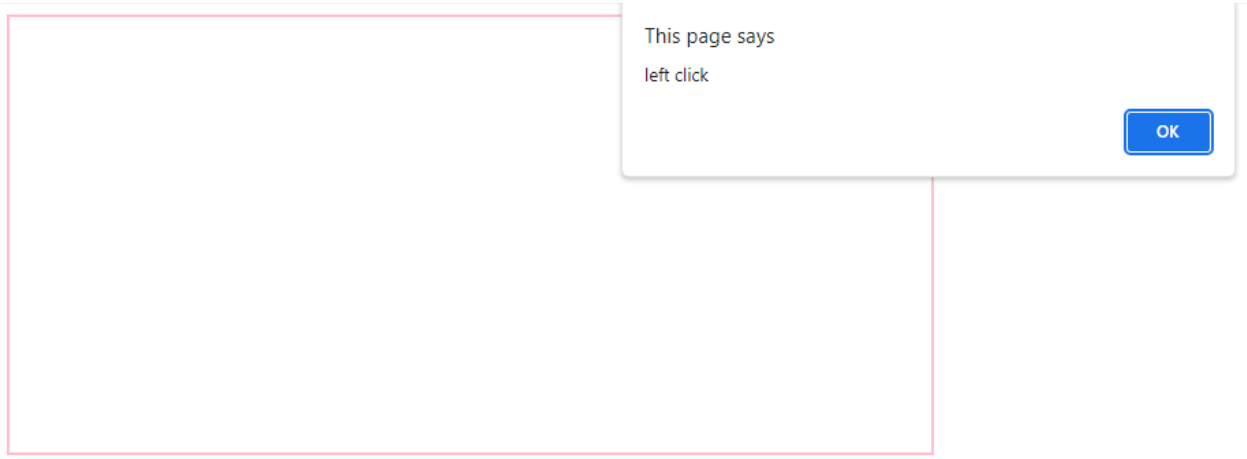


Example: Write a Js to check which mouse button is clicked (right, left or middle)

```
<html>
<head>
<script type="text/javascript">
function fun(e)
{
   val = e.button;
   switch(val)
   {
      case 0: alert("left click");
      break;
      case 1: alert("middle click");
      break;
      case 2: alert("right click");
      break;
    }
}
</script>
</head>
<body>
```

```
<div style="width:50%;height:50%; border:2px solid pink;" onmousedown="fun(event)"></div>
</body>
</html>
```

Output:



Example (onkeyup, onkeypress, onkeydown)

```
<html>
<head>
<script type="text/javascript">
function fun(id)
{
id.bgColor="blue";
}
function fun2(e)
{
alert(e.keyCode);
}
function fun3(id)
{
```

```
id.bgColor="red";
}
</script>
</head>
<body onkeyup="fun(this)" onkeypress="fun2(event)" onkeydown="fun3(this)">
</body>
</html>
```

Output:

Check output after pressing shift key once and any letter key once.

Example: Write JS to handle following key events

1) Give keycode for the key pressed

2) Script should give message "vowel is pressed" on pressing vowel key

3) Background color should change to red after releasing the pressed key

```
<html>
<head>
<script type="text/javascript">
function fun2(e)
{
c= (e.key);
 if(c=='a' || c=='e' || c=='i' || c=='o' || c=='u' || c=='A' || c=='E' || c=='I' ||
c=='O' || c=='U')
 {
 document.write("vowel is entered "+e.keyCode+" "+ e.key);
 }
 else
 {
```

```
 document.write(e.keyCode+" "+ e.key);
 }
}
function fun3(id)
{
   id.bgColor="red";
}
</script>
</head>
<body onkeypress="fun2(event)" onkeyup="fun3(this)">
</body>
</html>
```

Example: Write JS to handle following mouse events

1) If mouse is over heading should turn yellow, If mouse goes out then it should turn black.

2) If find time button is clicked then show date and time information.

3) If button named "red" is clicked then background color should turn red, and button named "green" is clicked then background color should turn green.

```
<html>
<head>
<script type="text/javascript">
function fun(id)
{
      id.style.color = "yellow";
}
function fun2(id)
{
      id.style.color = "black";
```

```
}
function fun3(id)
{
      d = new Date();
      document.getElementById("demo").innerHTML = d;
}
function fun4()
{
     id=document.getElementById("bd");
     id.bgColor = "red";
}
function fun5()
{
      id=document.getElementById("bd");
      id.bgColor = "green";
}
</script>
</head>
<body id="bd">
     <h1 onmouseover="fun(this)" onmouseout="fun2(this)">Hello</h1>
     <input type="submit" value="Find Time" onclick="fun3(this)"/>
     <p id="demo"></p>
     <input type="submit" value="red" onclick="fun4()"/>
     <input type="submit" value="green" onclick="fun5()"/>
</body>
</html>
```

Ways to access form elements

obj = document.forms[“form_name”][“element_name”].value;

or

obj = document. form_name. element_name.value;

Example(onsubmit)

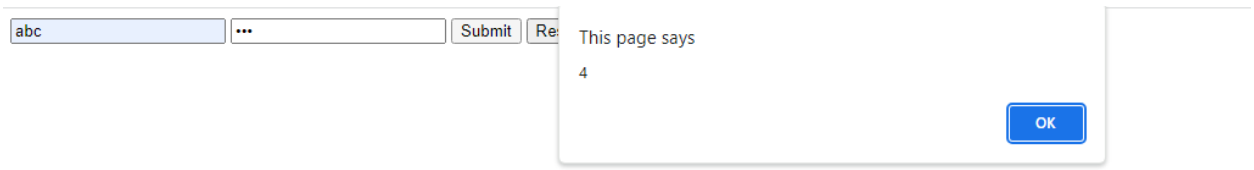
```
<html>
<head>
<script type="text/javascript">
function fun()
{
                // METHOD1
   obj2 = document.f1.t1.value;
   document.write(obj2);
            // METHOD2
   // obj3 = document.forms["f1"]["t1"].value;
   // document.write(obj3);}
</script>
</head>
<body>
<form name="f1" onsubmit="fun()">
   <input type="text" name="t1"/>
   <input type="password"/>
   <input type="submit"/>
   <input type="reset"/>
</form>
</body>
</html>
```

Example:

```
<html>
```

```
<head>
<script type="text/javascript">
function fun()
{
   obj = document.forms["f1"];
   alert(obj.length);
}
</script>
</head>
<body>
<form name="f1" onsubmit="fun()">
   <input type="text" name="t1"/>
   <input type="password"/>
   <input type="submit"/>
   <input type="reset"/>
</form>
</body>
</html>
```

Output: (on clicking submit button.)

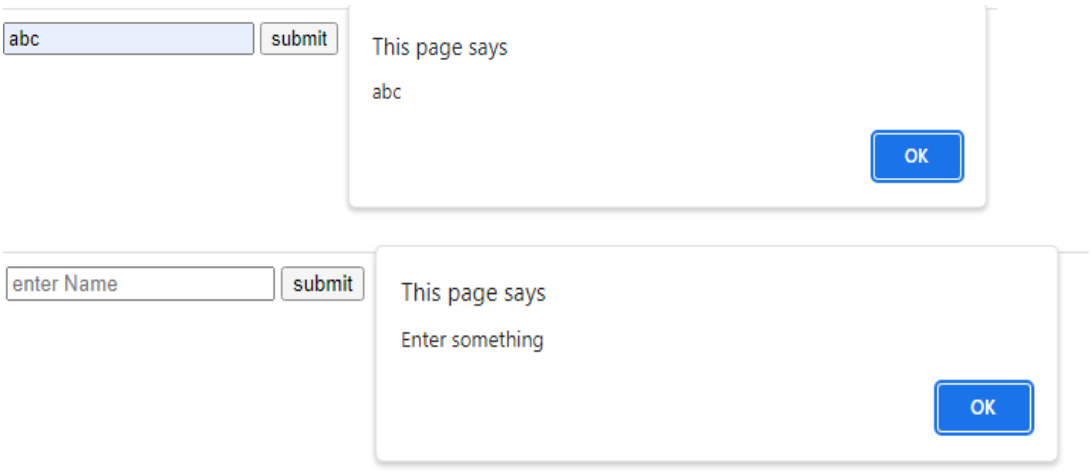


Example:

```
<html>
<body>
<script type="text/javascript">
function fun(demo)
```

```
{
    val=demo.t1.value;
    if(val=="")
    {
        alert("Enter something");
        return false;
   }
    alert(val);
}
</script>
<form action="#" onsubmit="fun(this)">
   <input type="text" maxlength="10" placeholder="enter Name" id="t1"/>
   <input type="submit" value="submit"/>
</form></body></html>
```

Output:



Example (another method of above example)

```
<html>
<head>
</head>
<body>
```

```
<script type="text/javascript">
function fun()
{
    val=document.getElementById("t1").value;
    if(val=="")
   {
        alert("Enter something");
        return false;
    }
    alert(val);
  }
</script>
<form action="#">
     <input type="text" maxlength="10" placeholder="enter Name" id="t1"/>
     <input type="submit" value="submit" onclick="fun()"/>
</form>
</body>
</html>
```

JavaScript Validation:

It is important to validate the form submitted by the user because it can have inappropriate values. So, validation is must to authenticate user.

JavaScript provides facility to validate the form on the client-side so data processing will be faster than server-side validation. Most of the web developers prefer JavaScript form validation.

Through JavaScript, we can validate name, password, email, date, mobile numbers and more fields.